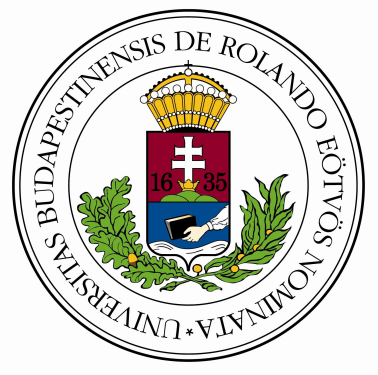
# Konkurens programozás

## Várakoztatók, explicit lock, condition




**Kozsik Tamás**

ELTE Eötvös Loránd Tudományegyetem

# Outline


1. Explicit lock
   - Író-olvasó szinkronizáció
   - Condition
2. Várakoztatók
   - A Future megértése

# Explicit zároló

```
package java.util.concurrent.locks;
public interface Lock {
    ...
    void lock();          // kritikus szakasz eleje
    void unlock();        // kritikus szakasz vége
}
```



# Explicit zároló

```
package java.util.concurrent.locks;
public interface Lock {
    ...
    void lock();          // kritikus szakasz eleje
    void unlock();        // kritikus szakasz vége
}
```

## A beépített (intrinsic) zároláshoz, a synchronizedhoz képest

- Többféle művelet
- Többféle lehetőség
- Kényelmetlenebb használat



# Tipikus használat

## Előkészítés

```
Lock l = new ReentrantLock();
```



# Tipikus használat

## Előkészítés

```
Lock l = new ReentrantLock();
```

## Kritikus szakaszok szálakban

```
l.lock();
try {
    // access the resource protected by this lock
} finally {
    l.unlock();
}
```



# Példa (1)

```
import java.util.concurrent.locks.*;
public class Account {

    private final Lock lock = new ReentrantLock();

    private int balance;

    public int balance(){
        lock.lock();
        try { return balance; }
        finally { lock.unlock(); }
    }

    ...
}
```



# Példa (2)

```
public void deposit( int amount ){
    assert amount > 0;
    lock.lock();
    try { balance += amount; }
    finally { lock.unlock(); }
}

public void withdraw( int amount ) throws InsufficientBalanceException {
    assert amount > 0;
    lock.lock();
    try {
        if( balance >= amount ) balance -= amount;
        else throw new InsufficientBalanceException();
    } finally { lock.unlock(); }
}
```



# Többféle művelet

## Nem blokkoló

```
boolean tryLock()
```

## Időhöz kötött

```
boolean tryLock( long time, TimeUnit unit )
                                    throws InterruptedException
```

## Félbeszakítható várakozás

```
void lockInterruptibly() throws InterruptedException
```



# Példa

```
import java.util.concurrent.locks.*;
public class Account {

    private final Lock lock = new ReentrantLock();

    private int balance;

    public int balance() throws InterruptedException {
        lock.lockInterruptibly();
        try { return balance; }
        finally { lock.unlock(); }
    }

    ...
}
```



# Többféle lehetőség

- Különböző metódusban a `lock()` és az `unlock()`
- Hand-over-hand locking
- Különböző viselkedésű megvalósítások
  - Pl. író-olvasó szinkronizáció
- Több *wait-set*-szerű várakoztatás

```
Condition newCondition()
```



# Író-olvasó szinkronizáció

- Readers-writers problem
  - Bizonyos (“olvasó”) tevékenységek egyidőben végrehajthatók
  - Bizonyos (“író”) tevékenységek kizárnak minden mást
- A kölcsönös kizárás gyengítése
- `java.util.concurrent.locks.ReentrantReadWriteLock`



# Példa (1)

```
import java.util.concurrent.locks.*;
public class Account {

    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private final Lock readLock = lock.readLock();
    private final Lock writeLock = lock.writeLock();

    private int balance;

    public int balance(){ ... }
    public void deposit( int amount ){ ... }
    public void withdraw( int amount )
        throws InsufficientBalanceException { ... }

}
```



# Példa (2)

```
private final ReadWriteLock lock = new ReentrantReadWriteLock();
private final Lock readLock = lock.readLock();
private final Lock writeLock = lock.writeLock();

public int balance(){
    readLock.lock();
    try { return balance; }
    finally { readLock.unlock(); }
}

public void deposit( int amount ){
    assert amount > 0;
    writeLock.lock();
    try { balance += amount; }
    finally { writeLock.unlock(); }
}
```



# Több termelő és több fogyasztó

- Nem tudjuk szabályozni, hogy a `notify()` melyiket ébressze fel
- Ugyanazt a kulcsot és *wait-set*et kell használni
- Mindenkit ébresszünk fel: `notifyAll()`



# Több termelő és több fogyasztó

- Nem tudjuk szabályozni, hogy a `notify()` melyiket ébressze fel
- Ugyanazt a kulcsot és *wait-set*et kell használni
- Mindenkit ébresszünk fel: `notifyAll()`

- Thundering herd problem



# Korlátos buffer

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();
    private final int capacity;
    public Buffer( int capacity ){ ... }

    public synchronized void put( T item ) throws InterruptedException {
        while( impl.size() == capacity ){ this.wait(); }
        impl.addLast(item);
        this.notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while( impl.isEmpty() ){ this.wait(); }
        this.notifyAll();
        return impl.removeFirst();
    }
}
```



# Explicit várakoztató: Condition

```
package java.util.concurrent.locks;
public interface Condition {
    void signal();
    void signalAll();
    void await() throws InterruptedException;
```



# Explicit várakoztató: Condition

```
package java.util.concurrent.locks;
public interface Condition {
    void signal();
    void signalAll();
    void await() throws InterruptedException;

    boolean await( long time, TimeUnit unit )
                                        throws InterruptedException
    long awaitNanos( long nanosTimeout )
                                        throws InterruptedException
    boolean awaitUntil( Date deadline )
                                        throws InterruptedException
    void awaitUninterruptibly();
}
```



# Korlátos buffer (1)

```
import java.util.List, java.util.LinkedList;
import java.util.concurrent.locks.*;
public class Buffer<T> {

    private final Lock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    private final List<T> impl = new LinkedList<T>();
    private final int capacity;

    public Buffer( int capacity ){
        if( capacity <= 0 ) throw new IllegalArgumentException();
        this.capacity = capacity;
    }

    public void put( T item ) throws InterruptedException { ... }
    public T take() throws InterruptedException { ... }
}
```



# Korlátos buffer (2)

```
    public void put( T item ) throws InterruptedException {
        lock.lock();
        try {
            while( impl.size() == capacity ){ notFull.await(); }
            impl.addLast(item);
            notEmpty.signal();
        } finally { lock.unlock(); }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while( impl.isEmpty() ){ notEmpty.await(); }
            notFull.signal();
            return impl.removeFirst();
        } finally { lock.unlock(); }
    }
}
```



# Outline


1. Explicit lock
   - Író-olvasó szinkronizáció
   - Condition
2. Várakoztatók
   - A Future megértése

# Várakoztatók

`java.util.concurrent`

- `Future`
- `BlockingQueue`
- `Semaphore`
- `CountDownLatch`
- `CyclicBarrier`
- `Phaser`
- `Exchanger`
- `SynchronousQueue`



# CountDownLatch példa (API docs) – 1

```
class Driver {
    void main() throws InterruptedException {
        CountDownLatch startSignal = new CountDownLatch(1);
        CountDownLatch doneSignal = new CountDownLatch(N);

        for (int i = 0; i < N; ++i) // create and start threads
            new Thread(new Worker(startSignal, doneSignal)).start();

        doSomethingElse();            // don't let workers run yet
        startSignal.countDown();      // let all workers proceed
        doSomethingElse();
        doneSignal.await();           // wait for all to finish
    }
}
```



# CountDownLatch példa (API docs) – 2

```
class Worker implements Runnable {
    private final CountDownLatch startSignal;
    private final CountDownLatch doneSignal;
    Worker(CountDownLatch startSignal, CountDownLatch doneSignal) {
        this.startSignal = startSignal;
        this.doneSignal = doneSignal;
    }
    public void run() {
        try {
            startSignal.await();
            doWork();
            doneSignal.countDown();
        } catch( InterruptedException ex ){} // return;
    }
    void doWork() { ... }
}
```



# Future interfész leegyszerűsítve

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    boolean isDone();
}
```



# Future interfész leegyszerűsítve

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    boolean isDone();
}
```

A `java.util.concurrent.Future` további műveleteket is tartalmaz.



# Használat

```
String[] sort( String[] inputs, ExecutorService exec )
            throws ExecutionException, InterruptedException
{
    String[] outputs = new String[inputs.length];
    Future<Integer>[] pending = new Future[inputs.length];
    Arrays.setAll( pending,
                   i -> exec.submit(
                                    () -> smallerOnes(i,inputs)
                                   )
    );
    for( int i = 0; i < inputs.length; ++i ){
        outputs[ pending[i].get() ]   =   inputs[i];
    }
    return outputs;
}
```



# Másik példa használatra

```
public static void main( String[] args ) throws InterruptedException {
   Future[] futures = new Future[args.length];
   Arrays.setAll( futures,
                  i -> Async.call( () -> {
                                     int n = Integer.parseInt(args[i]);
                                     Thread.sleep(n);
                                     return null;
                                   }
                                 )
                );
   for( Future future: futures ){
      try { future.get(); }
      catch( ExecutionException e ){
         e.getCause().printStackTrace();
      }
   }
}
```



# Future létrehozása: pl. aszinkron hívás

```
public class Async {

    public static <T> Future<T> call( Callable<T> task ){
        Promise<T> promise = new Promise<T>(task);
        promise.start();
        return promise;
    }
```



# Future létrehozása: pl. aszinkron hívás

```
public class Async {

    public static <T> Future<T> call( Callable<T> task ){
        Promise<T> promise = new Promise<T>(task);
        promise.start();
        return promise;
    }

    private static class Promise<T> extends Thread
                                    implements Future<T> {
        ...
    }

}
```



# Future egy lehetséges megvalósítása

```
class Promise<T> extends Thread implements Future<T> {
  private Callable<T> task;          private T result = null;
                                     private Throwable exc = null;
  Promise( Callable<T> task ){ this.task = task; }

  @Override public void run(){
      try { result = task.call(); }
      catch( Throwable t ){ exc = t; }
  }
  public T get() throws InterruptedException, ExecutionException {
      this.join();
      if( exc == null ){ return result; }
      else { throw new ExecutionException(exc); }
  }
  public boolean isDone(){ return isAlive(); }
}
```

